

# ***Adicionando Gerenciamento de Memória Virtual ao seu Simulador de Processos do T1***

## **Segundo Trabalho de Sistemas de Computação – INF 1316**

Prazo de entrega (envio por E-mail): 23 de junho, 23:59

Data de apresentação (na bancada): 25 de junho, 11-13 hs

### **Introdução**

No segundo trabalho você irá estender o seu Trabalho-1 adicionando o gerenciamento de memória virtual, sendo que as maiores modificações deverão ocorrer também no **KernelSim**, processo simulador do kernel/núcleo. Essas serão a implementação das *tabelas de página* TP, de cada um dos cinco **processos de aplicação** (A1, A2, A3, A4, A5), e o mapeamento de cada página lógica (valor da variável *mem* de Ax) para um **dos 32 quadros de página** de uma hipotética memória principal, chamada de RAM.

Você continuará usando também o controlador de interrupções (**InterContolerSim**), só que ele irá gerar, além dos IRQ0, IRQ1 e IRQ2 (como no Trabalho-1) também uma interrupção vinda supostamente de partição de swap hipotética, digamos, a interrupção **IRQ3**. Esta nova interrupção deverá sinalizar ao **KernelSim** quando uma operação de read/write do swap para a memória RAM, ou vice-versa, tiver terminado. Por exemplo, para buscar uma nova página lógica de um Ax e inseri-la em um quadro de páginas da RAM determinado pela política de substituição de páginas a ser implementada. Ou seja, a interrupção IRQ0 continua sinalizando o final de um TimeSlice, e as interrupções IRQ1 e IRQ2 indicam, respectivamente, o término da operação no dispositivos D1 e D2. A interrupção IRQ3, por sua vez, indica “término de uma operação da partição do disco de swapping”, e irá desbloquear o primeiro processo Ax na fila de espera (pelo espaço em memória).

### **Simulando os acessos à memória como system calls**

Como você viu nas aulas teóricas, a tradução do endereço lógico para o endereço físico de memória acontece a cada instrução de máquina gerada pela CPU, ou seja, é uma tradução automática realizada pela Memory Management Unit (MMU)<sup>1</sup>, e não tem nada a ver com system calls realizadas pelos processos. No entanto, para facilitar esse trabalho T2, “vamos fazer de conta” que cada vez que um processo Ax acessa uma página virtual nova, diferente da sua página virtual atual, ou seja, quando a variável *mem* muda de valor, é como se fosse uma chamada de sistema parecida com a syscall de E/S para um dispositivo, D1 ou D2. Tá ligado?

Assim como no Trabalho-1, cada processo de aplicação Ax executa um *loop* (de até MAX iterações) e agora, além de decidir aleatoriamente (com certa probabilidade), se requisita um syscall para leitura ou escrita no dispositivo Dx, usando `syscall(Dx, R/W/X)`, dessa vez cada Ax cada iteração deverá decidir, com uma probabilidade maior, se a variável *mem* trocará de valor, escolher aleatoriamente o novo índice de página virtual (troca do valor da variável

---

<sup>1</sup> uma componente eletrônica no no chip da CPU

*mem* para outro valor no intervalo [0, 15] ), e determinar se o acesso nessa nova *mem* será de escrita ou leitura.

Exemplo do código do Ax:

```
PC++;
mem = rand()%15; //initial logical page index in 0..15 (Mem[16])
repeat {
    sleep(0.5)
    if (m =rand()%100 < 39) { // 40% chance to access new logical page
        mem = m %15; //the new logical page index among 0..15 (Mem[16])
        if (mem % 2) Op = R else Op = W;
        // inform KernelSim of access(mem, Op) as syscall (M, mem, Op)

        if (d = rand()%100 +1 < 15) { // generate a random syscall to D1 or D2
            if (d % 2) Dx = D1 else Dx= D2;
            if (d % 3 == 1) Op = R
            else if (d % 3 = 1) Op = W
            else Op = X;
            // inform KernelSim of syscall(Dx, Op) to device D1 or D2
        }
        sleep(0.5);
    } until (PC > MAX);
```

## **Novo KernelSim**

O KernelSim, então, terá que ser estendido com o gerenciamento de memória virtual para cada um dos cinco processos Ax. Como a memória física RAM só tem 32 quadros de página e são cinco A1,..A5 disputando a memória RAM, cada um com 16 pávinas virtuais (16 possíveis valores da variável *mem*) é claro que cada Ax só terá algumas de suas páginas virtuais posicionadas em alguns dos 32 quadros de página de RAM. Por isso o novo **Kernel-Sim** deverá implementar e atualizar, constantemente as seguintes estruturas de dados:

- globalmente, um único array, RAM [32], representando a memória física RAM com 32 page frames e
- um array RAM-free[32] de booleanos para designar os frames livres em RAM[32]
- uma tabela de página, TPx[16], uma para cada processo Ax, indicando quais de suas páginas virtuais estão em qual índice de pageframe no array de RAM.
- Cada entrada de uma tabela de página TP[i] deverá ter vários campos, mas pelo menos os seguintes:
  - **valid/invalid** bit: se essa página está na memória RAM ou não (valid =0)
  - **frame**: índice (ou ponteiro) para o page frame onde está localizado na RAM
  - **modifyBit**: se a última operação que ocasionou a obtenção de um page frame da RAM foi uma escrita, então modifyBit=1, senão modifyBit=0
  - **when**: o valor do PC de Ax na ocasião do mais recente access(mem,op) quando mem conseguiu de um page frame da RAM

## **Passo a passo da tradução de memória lógica para física**

Então, ao receber um **access(mem,op)** de um processo Ax, o KernelSim:

1. Bloqueia o processo Ax (SIGSTOP)
2. Verifica na tabela de páginas TPx[mem] se a entrada mem está válida, isto é, contém um índice/endereço de quadro de página na memória física RAM, ou não.
3. Se a página mem estiver na RAM (bit válido), imediatamente desbloqueia Ax (SIGCONT) e coloca na fila de processos prontos;
4. Se mem não estiver na RAM, temos um Page-fault e faça:
  - a. Procure, em **RAM-free**, se existe um quadro de página dando sopa (= vazio). E se sim, atribui a TPx[mem].frame o endereço do quadro de páginas escolhido, desbloqueia Ax (SIGCONT);
  - b. Se todos os quadros da RAM estiverem ocupados com páginas de processos, chame o seu **algoritmo de substituição de página substitute()**. Este retornará um quadro da RAM a ser alocado para Ax.mem e a ser sobrescrito, digamos, RAM[zz].
  - c. Atualize então a tabela TPx[mem].frame com zz, o endereço do quadro de página retornado por **subs()** e insira a requisição (Ax,mem) no final da fila de espera (**Sw\_Queue**), aguardando por uma interrupção IRQ3 do disco de swap;
  - d. Então, quando essa requisição chegar ao início da **Sw\_Queue**, é como se o conteúdo da página mem tivesse sido lido da partição de swap e possa ser escrito na RAM[zz];
  - e. Mas existem dois casos:
    - i. se o conteúdo prévio de RAM[zz] tiver sido uma página apenas lida, devido à operação prévia de access(..,R), o recebimento de uma única IRQ3 basta para sobreescrever o quadro RAM[zz] com a página Ax.mem.
    - ii. Porém, se conteúdo prévio de RAM[zz] tiver sido uma página escrita, este conteúdo prévio teria que ser previamente atualizado no disco de swapping. Ou seja, demandaria, primeiro, a escrita do conteúdo anterior no swap e, só depois, a leitura da página Ax.mem do swap, ou seja, duas interrupções IRQ3.
    - iii. Não, nós não vamos de fato salvar o conteúdo anterior de RAM[zz] no swap (até porque não implementamos uma partição de swap de fato), mas iremos alongar a espera total da requisição Ax.mem. Como? Fazendo com que, nesse caso, a entrada Ax.mem na Sw\_Queue tenha um contador de espera e, caso a requisição receba o primeiro IRQ3, decrementará esse contador e voltará ao final da mesma fila Sw\_Queue. Só quando chegar ao início da fila Sw\_Queue com contador=0 e IRQ3 chegar no KernelSim, a paginação terá sido concluída.

Obs: Isso é ilustrado na Figura 1. Se a requisição de processo A1.m3 for colocada no quadro RAM[4] então será resolvida assim que essa requisição chegar ao início da fila **Sw\_Queue**. Mas se for colocada no quadro RAM[1] o conteúdo prévio (A4,m0) teria que ser salvo primeiro no swap. Nesse caso, na fila **Sw\_Queue** a requisição (A1.m3) volta para o final da fila e tem que esperar mais até sua vez.

## Figura ilustrativa:

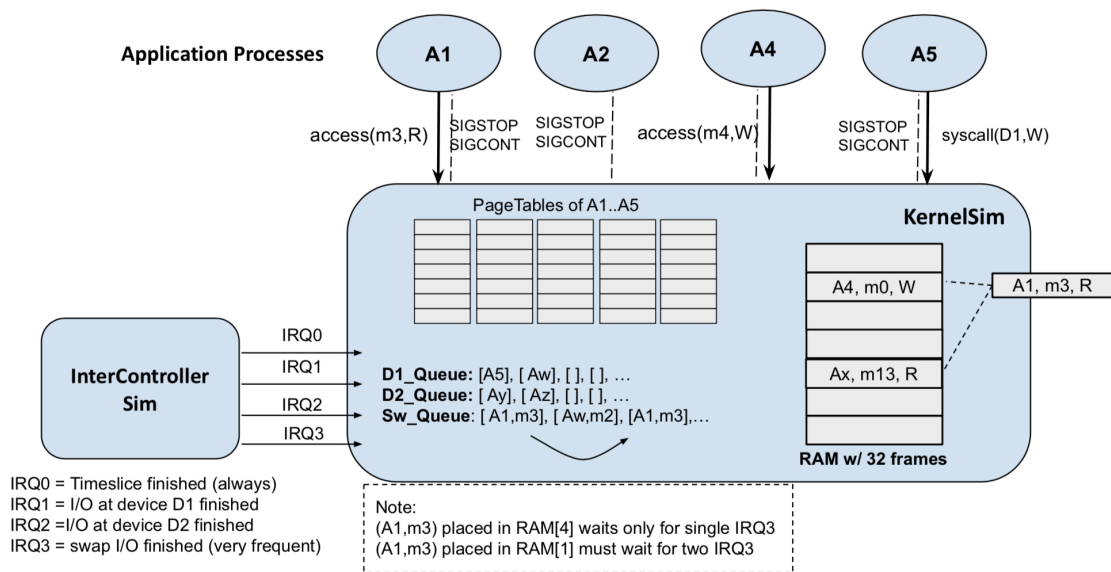


Figura 1: o KernelSim com estruturas necessárias para o gerenciamento da memória virtual.

## Escolha do quadro de página a ser sobreescrito: algoritmo substitute()

Como algoritmo de substituição de páginas (i.e., escolha do quadro de página a ser sobreescrito), você deve implementar duas rotinas e testá-las separadamente. Uma deve ser **int global\_substitute (RAM)** e a outra, **int local\_susbtitute(RAM)**, sendo que a “local” sempre irá selecionar um quadro de RAM ocupado com outra página do próprio processo Ax com acesso (Ax.nova\_mem), enquanto que a “global” permite que um processo “roube” uma página de qualquer outro processo Ay.

Além disso, você deve necessariamente usar a informação sobre conteúdos modificados (o bit M) de um quadro em RAM. De resto, baseie-se na aula sobre Substituição de páginas e escolha o algoritmo que você achar mais pertinente.

## O que não mudou do T1 para o T2

**Sete Processos:** No seu trabalho, cada um dos elementos em azul-cinza da Fig. 1 deve ser um processo Unix. Tanto **InterController Sim** como **KernelSim** devem ser processos infinitos que executam em paralelo.

**Sinais:** Para emular as interrupções (geradas por **InterControllerSim**) e tratadas por **KernelSim**, você pode usar qualquer mecanismo de comunicação IPC, como pipes, shm ou FIFO, e deve usar os sinais SIGSTOP e SIGCONT para permitir que o **KernelSim** pause e retome, respectivamente, a execução dos processos de aplicação A1,...,A5. Note que *access(mem,op)* também será considerada uma syscall (apenas para efeitos de simulação).

**Timesharing:** Qualquer processo Ax pode executar ininterruptamente, no máximo, por TimeSlice segundos. Portanto, quando chega um IRQ0 (Time Slice), KernelSim, deve enviar um SIGSTOP para o processo que estava executando, e escolher outro processo de aplicação Ay e

novamente ativar o mesmo usando o sinal SIGCONT, a menos que este processo esteja esperando pelo retorno de uma syscall (de E/S para D1, D2 ou Swap). Mais ainda, se dois processos, digamos Ay e Az, tiverem executado uma syscall para o mesmo dispositivo, deverão ser enfileirados na fila correspondente (D1/D2/Sw\_Queue) e liberados na ordem FCFS à medida que os IRQs correspondentes chegam ao **KernelSim**.

**InterControllerSim: Geração de Interrupções:** Em vez de simularmos um tempo específico que cada um dos dispositivos D1, D2 e Swap precisa para realizar os seus acessos de leitura ou escrita, faremos com que o **InterControllerSim** gere aleatoriamente interrupções para cada dispositivo (D1/D2, Sw) que indicam o término de qualquer pedido de E/S no dispositivo correspondente. A novidade é que você irá acrescentar uma nova interrupção, a IRQ3, proveniente do disco/partição de swapping.

Para isso, faça o **InterControllerSim** gerar, a cada 500 ms (usando sleep()) dentro do loop):

- Um IRQ0 com probabilidade 1 (frequência do TimeSlice)
- Um IRQ1 com probabilidade  $P_1 = 0.1$
- Um IRQ2 com probabilidade  $P_2 = 0.05$
- Um IRQ3 com probabilidade  $P_3 = 0.5$  (frequência de resposta da partição de swap)

Obs: Como no Trabalho-1, se você achar que essas frequências estão dificultando visualizar a execução, você pode ajustar a periodicidade do TimeSlice e as probabilidades de  $P_1$ ,  $P_2$  e  $P_3$ .

**Troca de Contexto:** No seu sistema, a troca de contexto acontecerá a cada vez que houver o chaveamento de um processo de aplicação para um outro. E, no caso específico do seu programa, esse contexto deve ser o contador PC do processo e a memória *mem* sendo acessada. Então, ao reativar um processo anteriormente interrompido, o seu contexto completo anteriormente salvo deve ser restaurado, em parte para garantir que todos os processos de aplicação Ax executem exatamente MAX interações. Além disso, se a interrupção de um processo se deveu a um syscall, então os parâmetros do syscall (D1 ou D2, R/W ou X) também devem fazer parte do contexto salvo e restaurado do processo.

**Visualizando os estados dos processos:** Além disso, seu simulador (envolvendo todos os processos incluindo **InterController Sim**), deve poder ser interrompido e reiniciado do ponto em que parou através Ctrl+Z *PID\_simulador*) e, quando estiver parado, deve mostrar as seguintes informações sobre todos os processos A1-A5:

- valor do PC e valor da última memória acessada (*mem*)
- em que estado está (estado pronto, ou bloqueado)
- se estiver bloqueado, em qual dispositivo e com qual operação
- se está executando (porque o **KernelSim** enviou um SIGCONT e ainda não um SIGSTOP para ele)
- Desde o início da execução e até esse valor de PC, quantas vezes o processo AP acessou cada um dos dispositivos.
- e se estiver terminado

- Além disso, o Kernel deve contabilizar o número total de page-faults causados por cada um dos processos A1..A5, e quantos dos tratamentos de page-faults tiveram que primeiro salvar o conteúdo de RAM[zz], e portanto, precisaram esperar dois IRQ3.

### **Observações Finais**

Como acontece a avaliação?

O trabalho pode ser feito individualmente ou em dupla. Deverá ser enviado na Data de Entrega (vide acima) para os e-mails do professor e do monitor, e deverá ser apresentado e explicado na Data de Apresentação na aula de laboratório (necessariamente com a presença da dupla). Cada dois dias de atraso acarreta um desconto de 1 ponto na nota máxima.

O que deve ser enviado/entregue?

Devem ser entregues o código-fonte e um relatório indicando que programas fazem parte do seu trabalho, incluindo eventuais programas de teste. Essa explicação também será objeto de avaliação.

### **Suete importante:**

Comece imediatamente. O trabalho é complexo e trabalhoso!!! Dá pra ver pelo tamanho do enunciado, certo?